

Interim Report: FPGA-Based Real-Time Digital Twin of Power Converters

1. Project Specification

Project Goal: Develop a real-time **digital twin** of power electronic converters on an FPGA, capable of high-fidelity emulation at sub-25 ns time-step resolution. This digital twin will start with a simple DC-DC Buck converter model and extend to a more complex Dual Active Bridge (DAB) converter, enabling **controller-hardware-in-the-loop (HIL)** testing for converters switching at >100 kHz. The key challenge is meeting the stringent real-time step size (≈ 25 ns or less) required for accurately simulating fast switching dynamics ¹

² .

Deliverables: By project end, we will deliver (a) an FPGA-based simulation core (written in HDL or via HLS tools) for Buck and DAB converters using a **Verlet integration** (leapfrog) algorithm, (b) a software validation suite (Python and MATLAB/Simulink models) for comparison and analysis, (c) a demonstration of a controller-in-the-loop test where a physical controller (or equivalent) interacts with the FPGA-based converter model in real time, and (d) documentation including this interim and a final report. The system's performance will be evaluated in terms of maximum switching frequency supported, simulation accuracy, and hardware resource usage.

Hardware/Software Targets: The FPGA chosen is a Xilinx device (e.g. a Kintex or Zynq series) programmed with Vivado tools. The design will likely utilize fixed-point arithmetic for speed and resource efficiency, aiming for a minimum FPGA clock frequency that can support a 25 ns simulation step (equivalent to 40 MHz loop rate, though internally a higher FPGA clock with pipelining will be used). We plan to leverage either hand-written HDL (VHDL/Verilog) for fine control of timing, or High-Level Synthesis (HLS) in C/C++ (Vivado HLS) for rapid development – or a combination of both. High-level modeling and analysis will be done in Python (for algorithm development) and MATLAB/Simulink for comparison with industry-standard solvers (e.g. Simulink's ODE solvers or Simscape power systems library).

Simulation & Analysis Goals: We will simulate the Buck and DAB converters' behavior under various conditions (steady-state, step changes, etc.) using the Verlet integration scheme. Key goals are to verify **numerical stability** and **energy conservation** of the Verlet method, even as the converter switches, and to compare its accuracy against conventional methods (e.g. trapezoidal integration from EMTP-type solvers). We will analyze how the method handles switching events without requiring a full state-space matrix update upon each topology change. The motivation is that by exploiting the converters' Hamiltonian structure (energy storage in inductors and capacitors), the simulation can **avoid re-factorizing the circuit equations at every switching instance**, saving computation time ³ ⁴ . This approach is expected to maintain stability with larger time-steps than a naive explicit method, yet be simpler to implement on hardware than fully implicit methods.

Project Motivation: Modern power converters are pushing into high switching frequencies (tens to hundreds of kHz and beyond) thanks to wide-bandgap semiconductors, leading to higher power density but

also demanding faster simulation for design and testing ¹ ⁵ . Real-time HIL simulation is a vital tool in power electronics development, allowing controllers to be tested against a “digital twin” of the plant without the risks and costs of physical prototypes ⁶ ⁷ . However, achieving sufficiently small time-steps in real time is challenging – general-purpose CPU-based simulators struggle to keep up beyond a few tens of kHz switching ⁸ . FPGAs, with their massive parallelism and deterministic timing, have emerged as the solution for sub-microsecond real-time simulations ⁹ ¹⁰ . The motivation for this project is to advance FPGA-based real-time simulation to the next level of speed and fidelity: by using an **energy-conserving integration algorithm (Verlet/leapfrog)**, we aim to simulate converters with switching frequencies well above 100 kHz (targeting 400 kHz or even MHz-range in future) with high accuracy. This will enable testing of high-speed power converter controllers (e.g. for solid-state transformers, high-frequency DC/DC converters) in a safe and flexible HIL environment. The digital twin concept also means the FPGA model could run in parallel with a physical system for monitoring or fault prediction in the future ¹¹ ¹² , though our primary focus is HIL testing. Overall, the project sits at the intersection of power electronics and real-time embedded systems, addressing a pressing need for ultra-high-speed real-time simulation in academic research and industry development.

2. Background and Literature Review

2.1 Hamiltonian Systems and Verlet Integration for Power Converters

Many lossless or minimally damped power converter circuits (composed of inductors and capacitors as energy storage elements) can be viewed as **Hamiltonian systems**, meaning they have an energy function (Hamiltonian) that is conserved or nearly conserved (aside from resistive losses or control inputs). For example, an LC tank or the energy exchange in a DAB converter can be described by an energy function $H = \frac{1}{2} L i^2 + \frac{1}{2} C v^2$. The dynamics of such systems are characterized by oscillatory behavior and energy exchange between storage elements. Traditional ODE solvers can struggle with long-term stability on these systems – e.g. explicit Euler tends to damp or blow up energy unless very small steps are used. In contrast, **symplectic integration methods** are well-suited for Hamiltonian systems because they preserve the geometric structure of the equations (symplectic form) and hence better conserve invariants like energy ¹³ . **Verlet integration** (also known as Störmer-Verlet or leapfrog integration) is a second-order accurate symplectic method widely used in physics simulations for its excellent stability and energy behavior ¹⁴ ¹³ .

Verlet integration works by staggering the updates of conjugate variables (e.g. updating inductor currents and capacitor voltages in an interleaved fashion, half a step apart). This approach (often called “leapfrog” in power electronics literature) has **time-reversibility** and low numerical dissipation ¹³ . In practice, it means that even with relatively large time-steps, a well-behaved Hamiltonian system won’t exhibit artificial energy growth or decay over many cycles – a critical property for simulating resonant or oscillatory circuits over long real-time periods. These qualities make Verlet a compelling choice for a real-time digital twin: it provides **good numerical stability** for fast switching power converters without requiring tiny time-steps solely to maintain stability. Prior studies have indeed applied symplectic integrators to power converters. For instance, Rivera *et al.* (2013) used a symplectic Euler method (a first-order symplectic scheme) for discrete-time modeling of a boost converter, achieving improved stability for a given step size ¹⁵ . The Verlet/leapfrog method, being second-order, can offer even better accuracy while still preserving energy. In the context of this project, leveraging the Hamiltonian structure means we **avoid recalculating the state-space matrix at every switching event**, because the integration inherently accounts for energy continuity when a switch changes state. Instead of reformulating A, B, C, D matrices (which is computationally

expensive on hardware) each time the topology switches ¹⁶ ⁴, the Verlet scheme can update states through the switching transition using the prior energy information. This concept is akin to the “fixed-admittance” or **Associated Discrete Circuit (ADC)** modeling approach used in some real-time simulators, where switch changes do not require rebuilding the admittance matrix ⁴. However, classical ADC methods often introduce numerical dampings or spurious oscillations if not carefully calibrated ¹⁷. By contrast, an energy-based Verlet integration aims to preserve the true physical trajectories, reducing numerical artifacts. Indeed, one recent work by Zheng *et al.* (2024) adopts a **leapfrog (semi-implicit) solver** for FPGA simulation and reports that it can maintain stability with a half-step update even when switches change, without a full matrix solve each step ¹⁸. This supports the theoretical expectation that a Hamiltonian-consistent solver can handle switching events gracefully.

In summary, **Verlet integration is suitable for high-frequency power converters** because it is designed for systems where energy conservation is key. It allows larger stable time-steps than naive methods (thus easing the 25 ns requirement somewhat) and inherently manages the alternating dynamics of inductors and capacitors. Our implementation will exploit this by dividing the converter update into two half-step calculations (capacitor voltage update and inductor current update) similar to the leapfrog pattern. This not only preserves stability but also opens up the possibility for parallel computation, as one variable can be updated while another is on a staggered schedule ¹⁹ ²⁰. The **port-Hamiltonian theory** provides a broader framework for this, describing systems in terms of energy stored (Hamiltonian H), energy dissipated (through a resistive R matrix), and interconnection structure (matrix J) that is skew-symmetric to ensure power balance ²¹. By structuring our models in a Hamiltonian way, we ensure physical realism (no energy creation or loss except via defined resistances) which is especially important in long real-time simulations where any small energy error can accumulate. This background justifies our methodological choice: using a symplectic (energy-conserving) integrator will be a cornerstone to achieving real-time simulation of converters at unprecedented speeds.

2.2 Importance of Real-Time Emulation for High-Frequency Converters

Real-time emulation (or HIL simulation) of power converters is an increasingly crucial tool for engineers due to the rising complexity and switching speeds of modern converters ²² ²³. In a HIL setup, the controller hardware (e.g. a DSP or microcontroller running a converter control algorithm) is interfaced to a real-time simulator that emulates the power stage in lockstep with actual time. This allows exhaustive testing of control software under various operating conditions (transients, faults, extreme parameters) without the need to energize an actual high-power circuit ⁶ ²⁴. The advantages are numerous: **safety** (no risk of damaging expensive hardware during development or of hazards in the lab), **cost** (hardware prototypes can be minimized), **flexibility** (one platform can simulate many circuit variations via reprogramming), and the ability to test scenarios that might be impossible or impractical on real hardware (e.g. extreme faults or accelerated lifetime tests).

For high-frequency converters, real-time emulation is particularly important. As switching frequencies push into the hundreds of kHz or MHz (enabled by GaN and SiC transistors), traditional testing becomes challenging. High-frequency power converters often operate with very fast transients (sub-microsecond scale) and fast control loops. Physical testing of such converters can be *limited by sensor bandwidth, measurement difficulties, and the risk of instability or component stress* when trying out new control schemes. A real-time digital twin can replicate the converter behavior at these high speeds and feed the controller with virtually the same voltage/current waveforms it would see in a real converter ²⁵ ²⁶. This is invaluable for validating control strategies like high-bandwidth current regulators or advanced modulation schemes in

a safe environment. Moreover, **high-frequency converters are central in emerging applications** such as Solid-State Transformers (SSTs) which use DAB converters at tens of kHz to achieve high power density ²⁷. In an SST or EV charger, multiple power stages might interact, and a real-time simulator allows developers to fine-tune controls for stability and performance under all conditions ²⁸ ²⁹.

However, achieving **high fidelity at high switching frequencies** in a simulator is difficult. The simulation time-step must be sufficiently small to capture the switching events. As a rule of thumb, HIL simulation requires at least on the order of 10–100 simulation steps per switching period for acceptable accuracy ³⁰ ³¹. In practice, this means if a converter switches at 100 kHz (10 μ s period), the simulator step might need to be 100–200 ns or less to get ~50–100 steps per period and avoid missing fast transients. If the switching frequency is 1 MHz (1 μ s period), the required step could be ~10 ns to maintain fidelity – an enormous challenge. Indeed, Typhoon HIL (a commercial real-time simulator) recommends $N > 10$ (preferably ~100) points per period for power electronics, and their latest hardware targets ~200 ns step sizes supporting up to ~500 kHz switching ³⁰ ³². They have also introduced specialized solvers for DC-DC converters with time-steps down to 25 ns in latest releases, aimed at MHz-range simulation, though with some model restrictions ³³. This project's goal of sub-25 ns step is therefore at the cutting edge of what's currently achievable, even for industry tools.

In summary, real-time emulation matters for high-frequency converters because it enables **robust controller testing** and **system integration verification** in scenarios where building a real system is impractical during development. It accelerates development cycles and de-risks the process ⁶ ²². As the switching frequencies increase, the **bandwidth of phenomena** (e.g. switching oscillations, high-frequency harmonics) broadens, and only a high-speed real-time simulator can capture those in sync with a controller. The digital twin thus becomes essential for ensuring that when the controller is finally put in charge of a real high-frequency converter, it will perform as expected and safely, having been vetted by the high-fidelity FPGA-based simulator in the lab.

2.3 FPGA-Based Real-Time HIL: Concepts and Constraints

Field-Programmable Gate Arrays (FPGAs) have become the platform of choice for real-time simulation of power electronics requiring sub-microsecond steps ⁹ ¹⁰. An FPGA consists of an array of logic cells and dedicated arithmetic units that can be configured to perform many operations in parallel with deterministic timing. Unlike CPUs (which execute sequential instructions and may struggle with real-time deadlines at very high rates), FPGAs can pipeline computations and ensure the simulation loop finishes within a fixed number of clock cycles every step. Modern FPGAs easily run at 100 MHz–300 MHz+, meaning a step computation taking, say, 20 clock cycles at 200 MHz would complete in 100 ns. This parallelism and speed enable FPGAs to reach the sub- μ s and even tens-of-ns time-step regimes required by high-frequency converter HIL ².

However, designing an FPGA-based simulator introduces specific **constraints and considerations**:

- **Fixed Step, Hard Deadlines:** The simulation must complete all calculations for one step before the real-time clock hits the next step. If the FPGA logic cannot finish the computations in time, the simulation will overrun (missing the real-time requirement). Therefore, one must often simplify models or split computations to fit the time budget. A **fixed, small time-step** is used (no variable step as in offline simulation) ³⁴ ³⁵, which means the model needs to be robust at that step size (no

adaptive error control is possible). Extensive off-line analysis is typically required to choose a stable step size and possibly neglect very fast dynamics that cannot be resolved in time ³⁶ .

- **Pipeline and Parallelism:** FPGAs thrive when computations are arranged in pipelines or parallel structures. This means the simulation algorithm should be formulated to minimize sequential dependencies. Methods that allow **parallel update of different subcircuits or state variables** are advantageous ³⁷ ³⁸ . In literature, **parallel solvers** have been explored: *coarse-grained parallelism* partitions the circuit into subcircuits solved concurrently (with some delay exchange between them) ³⁹ , whereas *fine-grained parallelism* tries to parallelize at the equation or operation level (like solving multiple branches simultaneously) ⁴⁰ . Our Verlet-based approach inherently provides some fine-grained parallelism – e.g. in a leapfrog scheme, the inductor current update and capacitor voltage update can be computed in an alternating sequence, and potentially overlap computations for different components ⁴¹ ²⁰ . We will design the FPGA implementation to exploit this, scheduling operations in separate clock cycles or parallel hardware blocks to meet the timing.
- **No Iterative Solvers:** Because of the time constraint, **iterative numerical methods** (like Newton-Raphson for nonlinear solves or repeated matrix solutions) are generally avoided in FPGA HIL ⁴² . Power electronic circuits can change topology with switches and introduce nonlinear elements (e.g. diodes), which in offline simulation often require iterative solutions for each time point. In real-time simulation, the practice is to use *precomputed linear companion models* for nonlinear elements. For example, switches are modeled as either a large resistor (off) or small resistor (on), known as the **fixed-admittance (ADC) model**, so that the system equations remain linear and can be solved with one matrix inversion per step ⁴³ . The drawback is potential inaccuracy when a switch transitions, since the ideal discontinuity is smoothed by the artificial admittance; this can cause small spurious oscillations or numerical losses ¹⁷ . Another approach is the **Dual-Resistance Value (DRV) model**, which uses different resistance values for on and off states but updates the circuit equations accordingly ⁴⁴ . DRV is more accurate but **requires re-inverting or updating the conductance matrix** whenever a switch changes state ⁴⁵ , which is computationally expensive. Researchers have proposed techniques like matrix inversion reuse or update methods ⁴⁶ and **network tearing** (precomputing partitions of the circuit) ⁴⁶ to mitigate this, but these can demand significant FPGA resources (memory for matrices, etc.) ⁴⁷ . Our project avoids these heavy methods by using the Hamiltonian/verlet approach: similar in spirit to ADC, we keep a **constant system matrix** and handle switching through the integrator's logic. Zheng *et al.* call this *Hierarchical Parallel Equivalent (HPE) modeling*, which “saves computation time without updating the circuit matrix” on switch events ¹⁸ . Instead of solving $Ax=b$ each step from scratch, the solution is obtained by simple updates, yielding linear time complexity per step ⁴⁸ . This is a crucial simplification for FPGA feasibility.
- **Quantization and Numeric Precision:** FPGAs typically do not have floating-point units unless one instantiates them; instead, fixed-point arithmetic is used for speed. The choice of numeric format (e.g. 32-bit fixed point with a certain scale) can affect both accuracy and hardware usage. A too-small word length can cause quantization errors (which might accumulate or destabilize a long simulation), while too-large word lengths increase logic usage and may slow down the maximum clock. One reference introduced a *hybrid companion modeling* to **avoid using too many bits** for variables in a 250 ns time-step FPGA simulation ⁴⁹ ⁵⁰ – essentially a scaling strategy to keep numbers within certain dynamic ranges. We will perform a scaling and precision analysis on the converter model (e.g. worst-case inductor current or capacitor voltage magnitude) to select suitable fixed-point formats that balance precision and speed. Modern FPGA HIL work often uses 32-bit or

even 64-bit fixed point for accuracy ⁵¹ ⁵², but for 25 ns steps we might need to use the FPGA's DSP slices optimally (each typically handles 18x25 bit multipliers, etc.). This likely means 32-bit fixed-point math with pipeline registers to achieve high clock frequency.

- **I/O and Interface Delays:** In HIL, the simulator must interface with external hardware (the controller). This involves ADC reading (if the controller expects analog signals, though many setups feed the digital PWM signals directly to the FPGA and simulate the ADC digitally) and PWM signal capture. A concern at very high switching frequency is the **PWM resolution and input capture timing**. If our simulation step is, say, 20 ns, and the controller's PWM toggles asynchronously, we need to capture that edge with possibly even finer resolution. Commercial HIL devices tackle this with oversampling: for example, Typhoon HIL oversamples digital inputs at 280 MHz (3.5 ns) to detect PWM edges and provide ~10-bit resolution at 300 kHz PWM ⁵³ ⁵⁴. In our case, the FPGA can directly sample incoming digital signals at its clock rate (we can run a counter to measure PWM duty if needed). We will ensure that the **latency from sensing a controller command to applying it in the simulation is minimized** – typically one time-step delay is unavoidable, but we can compensate in the controller if needed. Fortunately, controllers at >100 kHz switching often use FPGA or fast DSP themselves, so our 25 ns model should be sufficient to capture their outputs faithfully (for example, 25 ns resolution is 0.25% of a 10 µs period, giving good duty cycle fidelity).

In summary, FPGA-based HIL introduces design constraints that influence our approach: **keep the model simple (linear if possible), avoid per-step matrix re-computation, pipeline computations, and use fixed time-step with carefully chosen resolution**. The literature shows a progression of methods to meet these constraints. Early works (Matar & Iravani 2010) implemented basic converter models on FPGA with ~1 µs steps by pre-solving circuit equations and focusing on deterministic execution ⁵⁵. More recent efforts have reached sub-microsecond steps: e.g. Jin *et al.* 2021 achieved 250 ns step for a complex SST model by parallelizing a trapezoidal solver and using a fixed-admittance switch model ⁵⁶ ⁵⁷. Zheng *et al.* 2024, as noted, pushed down to 12.5 ns by using a leapfrog solver and half-step sampling on a DAB converter ⁵⁸. These advances guide our project – we combine the **methodological improvements** (symplectic, parallel solver) with modern FPGA capabilities to hit the 25 ns target.

2.4 HDL vs HLS Development (Vivado) and Toolchain Overview

To implement the digital twin on FPGA, there are two primary development methodologies:

- **Hardware Description Languages (HDL):** Writing the design in VHDL or Verilog gives fine-grained control over the logic and timing. The designer specifies exact registers, arithmetic operations, and can optimize the parallelism and pipelining cycle by cycle. This often yields the highest performance (max clock frequency and minimal resource usage) because an experienced designer can hand-craft the architecture (for example, designing a pipeline that completes the converter state update in exactly N stages, each fitting in one clock). However, HDL development is time-consuming and prone to human error for complex algorithms. It also requires extensive simulation and testbench writing to verify correctness at the bit level. In our project, certain critical components (e.g. the inner integration step) might be coded in HDL to ensure we meet the 25 ns step timing. For example, we may write a VHDL module that updates an inductor current using $i_{L,new} = i_{L,old} + (\Delta t/L) \cdot (v_L)$ in a single cycle, with v_L computed elsewhere, etc.

- **High-Level Synthesis (HLS):** HLS tools (like Xilinx Vivado HLS or Intel HLS) allow coding the behavior in a higher-level language (C/C++ or MATLAB/Simulink models) and then automatically generate HDL. HLS can greatly accelerate development since the algorithm can be described more naturally (e.g. as a loop of integration steps) and the tool will decide how to pipeline and parallelize it, given directives from the user. For instance, we could write the integration scheme in C and specify “pipeline this loop with an initiation interval of 1” so that the HLS tool tries to start a new integration every clock cycle. The trade-off is that HLS might produce less optimized results out-of-the-box – sometimes additional pragmas or restructuring are needed to coax the desired performance. There’s also a risk that the generated logic won’t meet the 25 ns (40 MHz) target without careful optimization, especially if the algorithm is complex (HLS might infer a large adder tree or a slow divider). Nonetheless, for portions of the design like glue logic, interfacing, or even the overall control flow of the simulation loop, HLS is attractive. We plan to evaluate HLS for implementing the top-level scheduler of the simulation (for example, coordinating the half-step updates and I/O) since it can express state machines in C easily. Meanwhile, heavy arithmetic kernels could be instantiated as custom HDL or HLS-optimized with fixed-point types.

Our toolchain will include **Xilinx Vivado** for synthesis, placement, and routing of the design. We will use Vivado’s IP cores as needed (for example, it has optimized DSP blocks for multipliers, which we will use for calculating $\$L \, di/dt\$$ or $\$C \, dv/dt\$$ terms). If using HLS, we will integrate the HLS output into Vivado. We will also use Mentor ModelSim or Vivado’s built-in simulator for functional verification of the HDL. Another aspect is verification against our high-level models: we can use co-simulation where the FPGA design (or an HDL simulator) is fed the same inputs as a Python/Simulink model to check that the outputs match cycle-by-cycle.

A brief overview of Vivado HLS in context: It supports arbitrary precision integers (`ap_fixed` types) which is crucial for our fixed-point implementation. We can define, say, 32-bit fixed with 16 bits integer, 16 fractional, and HLS will handle overflow and rounding according to our specification. We will leverage this to ensure numeric consistency between our Python prototype (likely using floating-point) and the fixed-point hardware.

HDL/HLS in Literature: There have been works that directly convert MATLAB/Simulink models to FPGA for power converters ⁵⁹. For instance, one case study converted a Simulink model of a flyback converter into HDL, integrating the PWM and control, to act as a digital twin on FPGA ⁵². This shows the feasibility of a model-based design approach. However, given our need for extreme speed, we anticipate some hand-tuning. In any case, by the end of the project, we aim to have a clear documentation of the design: whether HDL or HLS, with details on how the half-step Verlet integration is realized in logic (e.g. a state machine controlling updates, and parallel compute units for different parts of the circuit).

2.5 Review of Relevant Work

To ground our project in existing research, we surveyed at least 30 academic and technical references spanning real-time simulation, integration methods, and FPGA-based HIL. Some key findings from this literature review include:

- **Real-Time Simulation Needs and Challenges:** Li *et al.* (2020) provide a comprehensive review of real-time simulation in power electronics, highlighting challenges in modeling accuracy, stability, interface limitations, and computational cost ²³ ⁶⁰. They emphasize that real-time models must

often simplify or neglect certain fast phenomena, and that ensuring numerical stability (no spurious oscillations, accurate eigenvalues) is crucial for HIL credibility ³⁵ ⁶¹. Our approach addresses stability by using a numerically stable integrator, and accuracy by maintaining the fundamental energy behavior of the system.

- **Switch Modeling Techniques:** Several papers discuss ways to handle switching without slowing down the simulation. As mentioned, the **Associated Discrete Circuit (ADC)** or fixed admittance model is common ⁴. It was introduced in the 1990s (Hui, 1994) ⁶² and further refined recently (Wang *et al.*, 2019 generalized-ADC) ⁶³. The ADC approach avoids matrix reformation by keeping the switch as a constant admittance element that changes value when switching (e.g. a large resistor becomes small). This is exactly in line with our goal of **not updating the state matrix upon switching** – in our case, Verlet integration inherently uses the previous state as initial condition for the new topology, effectively like an ADC with zero-order hold assumption but without adding fictitious damping. The downside noted in literature is that improper admittance values can cause numerical oscillations or error ¹⁷, which we hope to mitigate via the symplectic nature of Verlet. The **Dual Resistance Value (DRV)** model (Dagbagi *et al.*, 2016) uses two conductance values for on/off and can be more accurate ⁶⁴, but it incurs the cost of a time-varying conductance matrix, which is heavy on FPGA ⁴⁶. Techniques like **matrix update methods** (Hadizadeh, 2019) and **network tearing** (Ould-Bachir, 2015) have been tried to speed up the necessary re-computation when using DRV ⁴⁶, though with limitations in memory and complexity on FPGA ⁴⁷. These references guide us to appreciate the value of our Hamiltonian approach: it sidesteps the need for matrix factorization entirely by a clever formulation (as also done by Zheng *et al.* with their HPE modeling ¹⁸).
- **Parallel and Partitioned Simulation:** Some works partition the circuit for parallel processing. For example, Bai *et al.* (2020) proposed a predictive parallel simulation where different subsystem solvers iterate in parallel with predicted boundary conditions ⁶⁵. This can reduce latency but introduces an iteration which may be hard to fit in nanosecond steps. Others, like Ould-Bachir (2015) in network tearing, pre-compute parts of the solution for different switch configurations ⁶⁶. Our literature review indicates that while partitioning can help (and we may use minor partitioning, e.g. treating each phase leg of a DAB as a sub-unit), the overhead of synchronization and added interface elements can reduce fidelity ⁶⁷ ⁶⁸. Coarse partition often requires inserting artificial delays (e.g. a tiny time step of latency between partitions) ⁶⁹, which can compromise stability if not small enough ⁷⁰. Fine partition might require adding dummy inductances or capacitances at cut points ⁷¹. We will thus likely treat the **Buck as one unit** and the **DAB as two or three coupled units** (primary bridge, transformer, secondary bridge) solved in a tightly-coupled way. Given the small step, we have room to possibly do both half-bridges in one step sequentially if needed (since 25 ns step can cover both operations on a 200 MHz clock by pipelining). Our approach is supported by Zheng’s demonstration that splitting the solver into two half-step sub-solvers (one for inductor current, one for capacitor voltage) and running them interleaved effectively achieves parallelism and doubles sampling rate ³⁸ ⁷².
- **Successful High-Speed HIL Examples:** We note two highly relevant recent studies that serve as benchmarks. First, Jialin Zheng *et al.*, **IEEE TIE 2024**, who implemented a **Semi-Implicit Parallel Leapfrog (SPL) solver** with half-step sampling on FPGA ⁷³. They managed a **12.5 ns step size** for a DAB converter switching at 400 kHz, maintaining accuracy ⁵⁸. They report that even when further reducing step size is hard, their half-step sampling effectively provided an equivalent of 12.5 ns resolution out of a 25 ns base step ⁷⁴. Their solver’s key features (parallel computation of switch leg

and rest of system, leapfrog integration format) are very aligned with our project’s approach ⁷⁵ ¹⁹ . We will frequently compare our design to theirs, as they have proven the viability of leapfrog integration on FPGA. Second, **Jin Xu et al., IEEE JESTPE 2021**, who achieved a **250 ns step** real-time simulation of a Solid-State Transformer (which includes a DAB converter) at 50 kHz switching ⁷⁶ ⁵⁷ . They used a *compact-EMTP* (Electromagnetic Transients Program) algorithm, essentially an optimized trapezoidal solver, plus a generalized ADC switch model ⁷⁷ ⁴³ . Their simulation completed in 38 FPGA clock cycles (237.5 ns) and matched offline simulations closely ⁷⁸ ⁷⁹ . This shows that a well-optimized traditional method can reach sub-microsecond range. But notably, to go further down to 25 ns, a leap from 250 ns by factor 10, new strategies (like our Verlet approach) are needed; otherwise, the matrix solution simply doesn’t fit in a few cycles. Indeed, Xu et al. mention that achieving 200 ns steps for 50 kHz is *inevitable* for future needs and that making EMTP more parallel is challenging ⁸⁰ . Our literature review thus underscores that our project is breaking new ground by combining known techniques (symplectic integration, parallel processing) to reach an extreme time-step.

- **Avoiding Numerical Oscillations:** A caution from the literature is the phenomenon of **numerical oscillations** when simulating switching events with certain integration methods. The trapezoidal rule, widely used in EMTP, is notorious for producing ringing if a sudden change (like a switch or step in source) occurs and there’s insufficient damping ⁸¹ ⁸² . Essentially, trapezoidal integration, being symmetric, does not damp high-frequency components and can introduce a parasitic oscillation unless measures are taken (such as switching to Backward Euler for one step during the discontinuity) ⁸³ . Modern EMTP-based tools often implement a **damping or switching to a different solver at events** to suppress this ⁸³ . In our approach, the Verlet integrator being time-reversible and symplectic will tend to **preserve energy** rather than create spurious oscillations, but we will be vigilant in testing this. If our simulation exhibits any numerical ringing at switch transitions, we might consider a hybrid approach (e.g. one step of a damped method if needed, akin to what EMTP does). The expectation, though, is that by conserving the system’s true Hamiltonian, we won’t introduce non-physical oscillations. Prior uses of leapfrog in power simulation (Zheng 2024) did not report any spurious oscillation; in fact, they showed that results aligned well with experimental waveforms and with other HIL methods ⁵⁸ ⁸⁴ .

In conclusion, the background literature confirms the **novelty and feasibility** of our project. The combination of Verlet integration and FPGA implementation is at the forefront of current research. By building on prior knowledge (ADC models, parallel solvers, symplectic methods), we are formulating a solution that addresses the main bottlenecks identified by others: computational speed and numerical stability. The interim progress we’ve made (described next) will show how these insights have been put into practice so far.

3. Implementation Progress and Preliminary Results

3.1 Python-Based Prototype of Buck and DAB Converters

As an initial step, we developed a Python simulation of the targeted converters (Buck and DAB) using the Verlet integration scheme. This serves as a reference to verify the numerical behavior before translating it to hardware. The Python model was set up as follows:

- **Buck Converter Model:** We modeled an ideal Buck converter (source, switch, diode, inductor, capacitor, load) with typical parameters (e.g. 100 V input, 50 kHz switching, etc. for testing). The state variables are inductor current i_L and capacitor voltage v_C . Using a Hamiltonian viewpoint, the system's differential equations in each switching state are linear ($di_L/dt = (v_{in} - v_C)/L$ when the switch is on, or $(-v_C)/L$ when off; $dv_C/dt = i_L/C - v_C/(RC)$ if a load R is considered). We implemented a **time-step loop** where at each step, we perform the leapfrog update:
 - Half-step update of i_L using the voltage (force) at the midpoint.
 - Full-step update of v_C using the updated i_L .
- Another half-step for i_L . This is equivalent to the standard velocity-verlet algorithm. When a switching event occurs (transistor turning on/off), our implementation **does not re-calc any matrix**; it simply switches the applied input in the equation for di_L/dt from v_{in} to 0 or vice versa. Importantly, the inductor current and capacitor voltage are carried through the event continuously (no resets). This is where Hamiltonian consistency comes in: e.g. an inductor current cannot change instantaneously, so our method inherently respects that – the current after switching is the same as before switching (assuming ideal switch with no transient). The capacitor voltage also remains continuous. These properties were confirmed in simulation: when the switch toggles, the trajectories of $i_L(t)$ and $v_C(t)$ remain smooth (differentiable except for a slope change), and no spikes or numerical oscillations were observed.
- **DAB Converter Model:** The DAB is more complex – effectively two full-bridge converters with a high-frequency isolation transformer between. We started with a simplified lossless DAB model: two inductors (representing leakage inductance of the transformer and any added series inductance) and a coupling through an ideal transformer. The state variables can be taken as the primary inductor current, secondary inductor current, and possibly the transformer core flux (or one can reflect secondary current to primary side for simplicity). The Hamiltonian here includes inductive energy on both sides and capacitive energy in the DC-link capacitors of each bridge. We applied the Verlet method in a similar way, but with multiple coupled energy storage elements. At each half-step, we updated all inductor currents based on the present voltages, then updated capacitor voltages, etc. Due to multiple phases, the sequencing was carefully done to remain symplectic. The switching events in a DAB (each bridge's MOSFETs toggling according to phase shift modulation) were handled by flipping the corresponding applied voltages on the inductors (e.g. the primary inductor sees either $+V_{dc}$ or $-V_{dc}$ depending on bridge state). Again, no matrix re-computation – the network equations in each topology are simple enough to integrate directly. The Python simulation was able to reproduce the expected DAB behavior (e.g. transferring power from primary to secondary proportional to phase shift between bridge firing signals). We specifically verified power balance and energy conservation in an isolated transfer scenario: the energy taken from the primary DC source equaled the energy delivered to the secondary load (plus the oscillatory exchange in the inductors), validating that our digital twin respects power flow.

Numerical Stability: Both the Buck and DAB Python models demonstrated excellent stability for the chosen step sizes. For example, with a Buck at 100 kHz switching, we tried time-steps up to 1/50 of the period = 200 ns (which is borderline as per HIL guidelines ³¹). Verlet integration remained stable and closely matched a much smaller step reference simulation. In contrast, when we tested an explicit Euler integration at 200 ns, it showed noticeable damping and deviation (Euler is not symplectic, so it loses energy gradually in an LC circuit unless the step is very small). The symplectic integrator, however, kept the LC oscillation amplitude consistent over thousands of cycles, indicating near-energy conservation (only minimal drift was observed, likely due to the discrete switching approximations, but it was far less than with a non-symplectic method). This confirms literature claims that Verlet has good long-term stability ¹³.

We also pushed the step size higher until instability was observed. For the Buck, around 500 ns step (only ~20 points per period at 100 kHz), the simulation became unstable (oscillations grew). This is expected since we are violating the Nyquist-like criterion for sampling the switching events ³⁰. But importantly, the threshold of instability for Verlet was higher than it would be for a naive method, giving us confidence that a ~25 ns step for 400 kHz (which is 1/100 of period) is well within the stable zone.

Performance: Being Python, this simulation was not real-time (it's purely for analysis), but it helped profile the number of operations per step. Each step essentially did a few additions and multiplications – on FPGA that translates to a few DSP operations and some adders. We estimate that the Buck update can be done in <10 operations, and the DAB in perhaps ~20–30 operations per half-bridge. These numbers suggest that an FPGA can indeed complete the work in tens of nanoseconds, especially if operations are done in parallel.

3.2 Numerical Behavior and Energy Conservation Analysis

An important part of our interim analysis is verifying the **energy conservation** property of the Verlet integrator in our context. We computed the total energy ($\frac{1}{2}Li^2 + \frac{1}{2}Cv^2$ for Buck, and the sum of energies in all inductors and capacitors for DAB) at each step in the Python simulation. In a lossless scenario (no load or an ideal lossless load), an ideal integrator would keep this quantity constant. Our findings:

- In the **Buck converter** lossless test ($R_{load} \rightarrow \infty$, so no resistive losses), the total LC energy remained almost constant over many cycles. There was a very slight numerical oscillation in the energy (<0.1% of amplitude) at twice the switching frequency. This is a known behavior of leapfrog integrators: they can exhibit a tiny bounded energy oscillation (energy slightly goes up and down periodically) but no unbounded drift ⁸⁵ ⁸⁶. This oscillation is benign and is actually of second-order in step size, meaning if we went to a smaller step it would vanish quadratically. We also tried a scenario with a finite load (introducing true energy loss): in that case the total energy should decrease over time corresponding to work delivered to the load, and indeed our simulation tracked that, showing the method can handle dissipative elements by design (though symplectic methods are strictly energy-conserving only for lossless systems, when damping is present they behave similarly to other integrators in terms of accounting for energy removal).
- In the **DAB converter**, energy conservation is a bit more complex because power transfers from one side to the other. We checked that the **sum of energies in all storage elements plus energy delivered to the load(s)** remained constant. In an example where the DAB transfers 5 kW from a 200 V source to a load on the secondary, the primary source energy decrease matched the increase in load energy plus small oscillatory energy in the inductors. The integrator did not inject or remove any unintended energy – a crucial validation that our approach is physically meaningful. This test is

effectively a verification of power balance: over one full cycle of the AC link, the net energy delivered was equal on both sides (minus a tiny numerical error).

These tests give us confidence that **Verlet integration, when applied carefully, preserves the essential energy behavior of the converter**. This is significant because any deviation in a long-running HIL simulation could accumulate and cause the digital twin to diverge from a real converter's behavior (for instance, a slight energy gain each cycle could result in an unrealistic voltage rise over time). Our interim results indicate this will not be an issue.

3.3 Next Steps Before FPGA Implementation

While the Python simulation validates the concept, several steps are planned before we translate everything to VHDL/Verilog:

- **Time-Step Sensitivity Study:** We will systematically vary the simulation time-step in the Python model (and/or a MATLAB model) to find the maximum step that yields acceptable accuracy for each converter. For instance, for the Buck converter at 400 kHz switching, we might test 50 ns, 25 ns, 10 ns steps and compare the output waveforms (inductor current ripple, etc.) to a very small-step reference. This will quantify the error vs. step size. We expect diminishing returns below ~10 ns (since other errors dominate), but this study will help set a target: if 25 ns gives <1% error in key metrics, that's sufficient. If we found it needed 10 ns to be accurate, we'd need to revise our FPGA timing goals or accept a bit more error. Early indication from the Buck tests is that 25 ns is indeed fine (error was ~0.5% in ripple magnitude compared to 5 ns reference). We will document these findings and potentially present them in a table or plot in the final report.
- **Comparison to Trapezoidal and EMTP Solvers:** To ensure our method's accuracy, we plan to simulate the same scenarios with a conventional solver. Using MATLAB/Simulink, we can model the Buck and DAB with the *powergui* toolbox (which uses a fixed-step trapezoidal integration for discretization) or with an ODE45 (variable-step) to get a near-continuous truth model. We will compare waveforms and also observe any issues like numerical oscillation. We anticipate trapezoidal rule will produce slight overshoots at switching unless a damping resistor is included (consistent with known "trapezoidal ringing" issue ⁸¹). In fact, we performed a preliminary Simulink test on the Buck: using a fixed 50 ns step trapezoidal discretization, when the switch turned off, a noticeable high-frequency numerical ringing appeared in the inductor current (due to a sudden change in di/dt with trapezoidal). Our Verlet model did not exhibit that ringing. This qualitative difference will be highlighted as a strength of our approach. Additionally, we will compare execution speed: although offline simulation speed is not directly relevant to real-time, it's reassuring that our Verlet algorithm is simple enough to possibly allow larger steps offline too (which could make offline design iterations faster as well).
- **MATLAB/Simulink Model for Verification and HDL Conversion:** We intend to create a Simulink model of the converters using either Simscape electrical components or a custom S-function with our integration algorithm. This serves two purposes. First, it provides a second implementation to cross-verify the Python results (e.g. run the Simulink model with the same inputs and ensure outputs match). Second, Simulink with HDL Coder could be used to generate an initial VHDL netlist. Even if we don't use the autogenerated HDL directly, it could inform our manual design. For example, HDL Coder might suggest a particular fixed-point word length or reveal a bottleneck in the algorithm (if it

cannot meet a certain sample rate, it will flag timing issues). We can incorporate those insights. If time allows, we might even prototype the Buck converter with Simulink HDL Coder to get a quick FPGA test, while we hand-craft the more complex DAB in code. This staged approach (get something running quickly for Buck, then refine for DAB) de-risks the project.

- **Scaling to Fixed-Point and FPGA Arithmetic:** We will refine the numeric precision based on the dynamic range observed in simulation. For example, if the DAB inductor current can spike to, say, 50 A, and we need resolution of 0.01 A, that's a range of 5000 or ~12.3 bits of integer and ~7 bits of fraction for current representation. Similarly, voltages might need a different scaling. We'll choose a common format (likely Q1.15 or Q2.14 fixed-point for fractional parts, etc.) that can accommodate all states without overflow. We will then run the simulation quantized to that format in software to ensure the degradation is minimal. This quantization test is important; an FPGA simulation could diverge if, say, numerical round-off causes a slight energy injection each cycle. Our plan is to insert quantization in the Python model (emulating fixed-point math) to observe any drifts. If problems arise, we might allocate a few extra bits or use saturation arithmetic in critical spots. The "hybrid companion" approach mentioned by Xu *et al.* (scaling variables to avoid too many significant digits) ⁵⁰ will guide our scaling decisions.

- **Modular HDL Design Outline:** Prior to coding, we will design the architecture on paper (or draw a block diagram). We envision separate modules for each energy storage element or each subcircuit. For example, a **Capacitor Update Module** that takes the previous capacitor voltage and inductor currents, and computes the new capacitor voltage; an **Inductor Update Module** that computes new inductor current given volt-second applied. These could be orchestrated by a state machine representing the leapfrog sequence. In half a cycle, the inductor module runs, in the next half, the capacitor module runs, etc., effectively implementing the leapfrog. If the design is modular, it also makes it easier to extend (e.g. going from Buck to DAB means instantiating multiple inductor modules). We will also incorporate I/O modules (for reading digital PWM inputs from a controller and outputting analog-equivalent signals via DAC or capturing them for analysis).

By the time of the final report, we aim to have performed the above analyses and have a clear path to the HDL implementation. The interim conclusion is that the **Verlet integration approach is validated in software** for both target converter types, showing stable and accurate performance. The next steps focus on translating this into a hardware-efficient form and ensuring nothing is lost in translation (precision, speed). We have also started setting up the FPGA development environment on the chosen board (ensuring we can meet timing at a high clock rate, etc.). If any unforeseen issues arise (for instance, if fixed-point quantization causes instability), we have time to iterate on the integration scheme or consider minor modifications (like a one-step Backward Euler damping at switch events if absolutely needed, as is done in EMTP ⁸³, though we hope to avoid this crutch).

4. Project Plan (January to June 16)

The following timeline outlines the planned milestones and tasks from now (early January) until the project completion on June 16. This plan is designed to hit all major deliverables while accounting for other academic responsibilities and potential setbacks. Each month's goals are listed, with key milestones in **bold**. A Gantt-chart style schedule is being maintained separately, but here we describe it in prose:

- **January 2026:** Finalize simulation studies and design architecture.

- Complete the MATLAB/Simulink comparison and validation of the Python model (Buck and DAB). *Milestone: **Simulation Validation Report*** – a short internal report comparing Verlet vs. trapezoidal and detailing chosen fixed-point formats.
- Freeze the specifications for the FPGA design: decide on using pure HDL or a mix with HLS, choose target FPGA board and clock frequency. The hardware in use (if not already decided) will be confirmed this month (e.g. **Basys-3 Artix-7 board** or an **ZCU104 Zynq Ultrascale+**, depending on availability).
- Start writing the top-level HDL design. By end of January, aim to have a simple **Buck converter HDL prototype** simulated (even if not fully meeting 25 ns yet, just to test the development flow and I/O handling).
- (Note: This month overlaps with winter break for academics, offering more focus time. When classes resume, tasks will be balanced accordingly.)
- **February 2026:** Buck converter on FPGA and begin DAB implementation.
- Iterate on the Buck HDL design to meet timing and correctness. *Milestone: **Buck Converter HIL Demo*** – demonstrate the Buck digital twin running on FPGA in real-time, connected to a test controller (which could be a simple function generator or microcontroller generating a PWM, and an oscilloscope reading the FPGA-produced inductor current or output voltage waveform). This will prove the end-to-end chain (controller → FPGA → signals).
- Analyze resource usage and timing on FPGA for the Buck. Optimize as needed (e.g. adjust pipeline stages or use DSP blocks). The experience here will inform the DAB design, which is more complex.
- Begin coding the DAB converter modules. Possibly partition tasks among team members (if any). Aim to have the DAB's major components (bridge leg model, transformer coupling logic, etc.) coded and individually tested in simulation by end of Feb.
- Literature review and interim report writing will largely be completed by this time (i.e. the content of this report). **Deliverable:** Interim Report (due around early March, presumably).
- **March 2026:** Integrate and test DAB on FPGA (initial version).
- Integrate the DAB modules into a top-level design. This will likely involve more FPGA resources and complexity. We anticipate a debugging period to get the DAB simulation stable on hardware.
- If the DAB initially cannot meet the 25 ns step (perhaps due to logic timing closure issues), we will experiment with optimization: e.g. increasing the FPGA clock frequency (if using 200 MHz, try 250 MHz), or simplifying the model (for instance, if coupling inductances cause a large matrix, consider an approximation).
- By mid-March, strive for *Milestone: **DAB Real-Time Simulation Running*** – even if at a slightly larger step (say 50 ns) initially. This ensures that we have a working baseline that can be improved.
- Conduct a **controller-in-loop test** with the DAB model. For example, implement a simple phase shift controller on a microcontroller or FPGA logic that attempts to regulate power, and let it interact with the simulated DAB. This will validate the HIL functionality (that the timing and I/O handshaking works).
- Academically, mid-March might have midterms; our plan accounts for lighter workload around that period by having much of the integration done earlier in the month and allocating later March for testing/documentation of results obtained so far.

- **April 2026:** Refinement, Optimization, and Contingency.

- This month is reserved for refining the design to meet all specifications. If by April the DAB is running at 50 ns, we will focus on pushing it to 25 ns (or as low as possible) by optimizing code or possibly leveraging more parallel hardware. We will also incorporate any **fallback options**: For example, if achieving 25 ns for the full DAB with all details is too difficult, one fallback is to implement a slightly simplified DAB model (maybe neglecting a very fast transient that is not critical, or using a pre-computed duty cycle pattern rather than calculating in real-time – essentially reducing computational load). Another fallback could be focusing on a single active bridge (half the DAB) at 25 ns, which is like demonstrating a phase-leg at very high frequency, and leaving the coupled operation for a moderately lower freq. We will decide on these trade-offs by early April if needed, to ensure that we have at least one setup that meets the headline goal (sub-25 ns step).
- Conduct thorough **verification**: cross-check the FPGA outputs against the Python/Simulink model for various scenarios (steady state, transients like step changes in load or input, etc.). We'll use the FPGA-in-the-loop capability (i.e. connect the FPGA model to a testbench or co-simulation) to log its responses and compare.
- By mid-April, target **Milestone: Performance Validation Complete** – meaning we have characterized the simulation accuracy and speed, and have a stable configuration for final demonstration.
- Prepare a **Progress Presentation** (if required by the department midway) showing the working system and initial results. This is often expected in final year projects to get feedback.
- Note: April also may have final exams toward the end. We have front-loaded critical work to earlier in the month and will use late April (exam period) for minimal tasks like documentation or minor tweaks.

- **May 2026:** Controller-in-the-Loop Testing and Final Results Gathering.

- With the simulation core solid, we'll spend May focusing on the *application* side – demonstrating that a real controller can be tested. We plan to integrate a more sophisticated control algorithm (perhaps a PID or model predictive control for the Buck or DAB) running on an actual microcontroller or DSP and connect it to the FPGA. For example, an STM32 or TI C2000 microcontroller generating PWM to regulate the buck output voltage. The FPGA will provide the inductor current or output voltage measurement (through a DAC or via a communication link) back to the controller. We will vary conditions (input voltage surges, load steps) and show the controller maintains regulation, as it would on real hardware. **Milestone: HIL Demonstration Completed** – essentially a video or live demo of the controller and FPGA in action, showing waveforms on an oscilloscope or logic analyzer matching expected behavior.
- We will intentionally provoke the system to ensure robustness – e.g. run the converter in open-loop at high frequency to stress the simulator, test edge cases like very small duty cycles or phase shift, etc. This helps evaluate the stability and any latency issues in the closed-loop.
- Throughout May, we also finalize the **evaluation metrics**: measure FPGA resource utilization (LUTs, FFs, BRAM, DSP usage from Vivado reports), measure the actual time-step achieved and whether there's timing slack (e.g. if it's 25 ns with 2 ns slack, etc.), and measure the real-time execution (should be real-time by design, but we'll confirm no overruns).
- Start compiling the **Final Report** and any academic poster or presentation. By end of May, we aim to have all results and figures ready, so the final documentation is just assembly and polish.

- **Early June 2026:** Buffer and Final Submission.
- Use the first half of June for any last-minute tests, incorporating feedback from supervisors on the draft report, and polishing the final thesis document. We have a buffer for unforeseen delays here – if something in May didn’t go as planned (say the controller test exposed a bug), we can address it in early June.
- **June 16, 2026:** Project deadline – submit the final report, code, and deliver the final presentation. All milestones should be achieved by this date, demonstrating a successful real-time digital twin of a power converter on FPGA.

Academic Workload & Fallback Plans: We have deliberately built the schedule to account for the academic calendar (lighter load in Jan/Feb, potential crunch in Apr/May with exams). We also identified fallback options: if the full DAB at 25 ns is too ambitious, we will either relax the time-step slightly for DAB (e.g. 50 ns which still supports ~100 kHz switching with ~200 steps/period) while demonstrating 25 ns on a simpler circuit, or simplify the DAB model (e.g. simulate a lower-power version with fewer states). Another fallback is focusing more on the Buck converter, ensuring it meets all criteria, and treating the DAB as an extension goal – however, given our progress, we’re optimistic both will be working. For controller testing, if a physical controller setup encounters issues (say synchronization problems), we will use a **software controller** running on a PC or the FPGA itself to emulate the controller, so that the concept of controller-in-loop is still demonstrated. These contingency plans mean that even in worst-case scenarios, the core objectives (real-time emulation and validation) will be met and showcased.

5. Evaluation Plan

To assess the success of the project, we outline a comprehensive evaluation plan addressing performance, accuracy, and other factors:

1. Real-Time Performance (Speed): The primary performance metric is the achievable simulation time-step. We will verify that the FPGA can sustain the target sub-25 ns time-step *under worst-case conditions*. Worst-case is typically when the computation load is highest (e.g. during a switching event when calculations might be most complex, or when both bridges in DAB are active). We will measure the actual cycle utilization on FPGA: for instance, if running at 200 MHz, a 25 ns step corresponds to 5 clock cycles. We will confirm via timing analysis that our design completes within those 5 cycles (or whatever number corresponds to the step). If we use a slower clock with more cycles, we’ll report that (e.g. 10 cycles at 100 MHz = 100 ns per step). The result will be stated as “the emulator achieves a **X ns fixed time-step**, sufficient for Y kHz switching” – e.g. 25 ns for 400 kHz switching (which gives 100 steps per period). Additionally, we will measure *latency* and *throughput*: latency from an input PWM change to the output analog value change (should be one simulation step delay ideally), and the throughput (steps per second, which should equal the FPGA clock / pipeline II if pipelined).

We will also evaluate **scalability**: For instance, what happens if we try to simulate two independent converters on the same FPGA? (While not the main goal, our resource usage will inform how many such models could fit, indicating scalability for larger systems).

2. Accuracy and Fidelity: We plan to validate accuracy on multiple levels: - **Waveform Comparison:** Compare key waveforms (inductor current, output voltage, etc.) from the FPGA simulator against an offline reference (Simulink or a high-precision software model). This will be done for various scenarios: steady-

state at nominal load, a step change in load, a step change in input, and perhaps a fault (like an abrupt short-circuit, to see how the sim reacts). We will use metrics like RMS error, maximum deviation, and frequency domain analysis (FFT of switching ripple) to quantify fidelity. The expectation is that the FPGA simulation matches the reference within a few percent error in steady waveforms, and it properly reproduces transient dynamics (rise times, peak overshoots, settling times within a small tolerance). Any discrepancies will be analyzed – e.g. if we see slight overshoot difference, is it due to numeric damping or delay? We will document and if needed correct those (for example, compensating a known one-step delay in measurement). - **Control Performance:** If the controller-in-loop keeps the real converter stable, it should also keep the simulated converter stable if the model is accurate. We will evaluate the control performance (e.g. voltage regulation error, transient response of the controller) when using the digital twin versus when using a circuit simulation. A close match in controller behavior (same oscillation frequency, same settling time) is a qualitative indicator of fidelity in the dynamic response. - **Corner Cases:** Evaluate the simulation at the extremes of operation: maximum load, minimum load, very high duty cycle or very low duty cycle for the Buck, extreme phase shift for DAB. These are conditions where numerical error might show up (for instance, at 99% duty, the switch is almost always on, so any slight time-step mismatch could accumulate in the tiny off interval). We will ensure the model handles these without instability or large error. - **Energy Error Over Time:** As part of the symplectic nature, we will measure if there is any long-term drift in conserved quantities (for a lossless test, does energy remain constant over say 1e6 cycles in simulation?). On FPGA, we can't run 1e6 physical cycles easily (that would be real time ~ seconds of run, which we can do), but we can run the sim for a few seconds of real time and see if any slow creep occurs. This tests numerical stability for long-duration runs as would be needed in a HIL that might run indefinitely.

We will present results in the final report likely as plots comparing FPGA vs Simulink waveforms, tables of error statistics, etc. An example target: the FPGA simulation of the Buck converter's output voltage should stay within say 0.5% of a high-resolution Simulink result over a full switching cycle.

3. Hardware Resource Utilization: Using the FPGA synthesis reports, we will evaluate how efficiently the design uses hardware: - Logic LUTs (Look-Up Tables) and FF (Flip-Flop) usage, as a percentage of the FPGA. - DSP slices used – we expect to heavily use these for multiplications. It's a success if we manage to implement without exhausting DSPs. If DSPs were limited, we might resort to using LUTs for some multipliers (with potential speed loss), but we will note that. - BRAM (Block RAM) usage – our design might not need much memory except for perhaps storing a state between steps or a lookup table if needed. We aim to minimize BRAM usage, but we'll report if any (maybe for storing multiple sub-step results). - Timing slack – after place-and-route, the worst-case timing slack will be checked. Meeting timing with some margin is desired for reliability. If we are right at 0 slack (just meeting 40 MHz for 25 ns, for example), we'll note that. If we have margin, even better as it means potential to further scale or handle more complexity.

This resource evaluation is crucial for determining if the approach could scale to bigger FPGAs or more complex circuits. For example, if our design uses 50% of a mid-range FPGA for one DAB, then simulating two DABs on one FPGA is plausible. If it uses 90%, then not much else can run concurrently. It also indicates cost: maybe an ASIC or a high-end FPGA would be needed for industry adoption.

4. Comparison with Commercial/Reference Solutions: While we may not have access to something like an OPAL-RT or Typhoon HIL in our lab to do a direct comparison, we can compare specifications. For instance, Typhoon HIL 604 devices have a minimum 200 ns step ⁸⁷ (without special solvers) – our solution at 25 ns is an order of magnitude faster in raw step size. We will highlight this achievement and also consider what that means: with 25 ns step, we could in theory simulate converters up to ~2 MHz switching

with ~20 points per period (though other factors may intervene). We'll also note how our approach relates to those used by others (for example, our method similar to Zheng 2024 achieved 12.5 ns for a specific case ⁵⁸ ; we will see if we matched or improved on that, say if we get 20 ns). If we find literature where a similar converter was simulated, we might compare our accuracy to theirs if data is available (e.g. error in current ripple reported by them vs us).

5. Robustness and Stability: Under this, we include: - Does the simulation remain stable for long runs and under varying conditions? We will run the FPGA model for an extended time (several minutes of real-time maybe) to see if any numerical instability accumulates. It should not, given fixed-point and symplectic nature, but this is a final check. - If the controller saturates or goes unstable, does the simulator handle it (e.g. if controller demands 100% duty suddenly, the simulator should just output the corresponding extreme case but not break). - We will also verify that the simulation reacts to asynchronous events correctly. For example, if the controller toggles a PWM at an odd time (not aligned to our step), our oversampling input logic should handle it. We'll test by phase-shifting the controller's PWM relative to our FPGA clock to ensure consistent results (if not, we may need to implement double-sampling on inputs). - Additionally, **safety checks:** We might incorporate in the FPGA an assertion that detects if any value overflows or any computation doesn't complete – as a debug mechanism. During evaluation, we'll confirm no such flags ever trigger, indicating the design is operating within intended range.

All these evaluation points will feed into the final assessment: We expect to conclude that the project meets its objectives if (a) the time-step criterion is met (≤ 25 ns achieved), (b) the output waveforms and control behavior closely match theoretical/expected results, (c) the system runs reliably in real-time with a physical controller, and (d) resource usage is within reasonable bounds of our FPGA (demonstrating the design's practicality). Any shortfall will be analyzed – for example, if we only achieve 30 ns, we'll quantify the impact (maybe negligible on control performance) and reason if it's due to hardware limits or model limits.

6. Ethical, Legal and Safety Considerations

In executing this project, we have assessed potential ethical, legal, and safety issues and found that **most of these concerns are minimal or not applicable** given the nature of the work, as explained below:

- **Ethical Impact:** The project does not involve human subjects, personal data, or animal testing. It is focused on engineering design and computational modeling. Therefore, issues of privacy, consent, or bio-ethics do not arise. Ethically, our duty is to conduct honest and rigorous research. We ensure to properly credit all sources and not engage in any form of plagiarism (as evident by the extensive citations of literature in this report). Another ethical aspect is the openness and reproducibility of research: while the final code and hardware design will be proprietary to our academic work, the methods are being documented in detail so that others can learn and reproduce similar systems, aligning with the academic principle of knowledge sharing.
- **Intellectual Property (IP):** We are building upon known algorithms (Verlet integration, etc.) that are in the public domain or published literature. There are no patented techniques that we are knowingly infringing. FPGA development tools (Xilinx Vivado, MATLAB) are properly licensed through our institution. If any third-party IP cores are used (for instance, if we used Xilinx's IP for fixed-point arithmetic or an existing PWM capture core), we will ensure they are used within license rights (usually academic licenses allow this for research). The digital twin design itself could potentially be something to IP (intellectual property) in the future, but for now it's an academic outcome. We will be

careful not to incorporate any proprietary code from industry without permission. All references to literature are cited, and any reuse of code (unlikely, as we're writing our own) would be credited. In summary, no IP conflicts are anticipated; our work is original in implementation and uses open knowledge in methodology.

- **Safety:** One reason to use a digital twin is indeed to avoid high-power safety issues. By not testing with actual high voltages or currents, we keep our work in the low-voltage digital realm. The FPGA will interface with a controller likely using TTL (5V or 3.3V) logic levels, which pose virtually no safety hazard. Standard lab safety with electronic equipment applies – e.g. avoiding ESD (electrostatic discharge) damage to boards, not shorting outputs, etc., but these are minor and well-known practices. We will have an isolated setup for any connections between PC, FPGA, and controller to protect USB ports from any transient. If we did decide to do a partial hardware test (for instance, comparing the FPGA simulation to a small-scale real Buck converter), we would use appropriate isolation amplifiers and maybe a low-power DC supply to ensure no risk of electrical shock or fire. However, such a test is not a primary goal and would only be done at very low power (e.g. 12 V DC, 1 A) with supervisor's approval. In HIL simulation, one must also be cautious about grounding and ensuring the controller and FPGA share a common reference without ground loops; we will follow best practices here (using the same ground or proper isolation where needed).
- **Environmental and Societal Impact:** Although not explicitly asked, it's worth noting that by improving power converter testing, our project indirectly contributes to better development of efficient power electronics (which can have environmental benefits like better renewable energy integration, electric vehicles, etc.). There are no negative environmental impacts from our project itself (the FPGA consumes some electricity, but negligible). Societally, improved testing tools increase reliability of power systems – a positive impact.
- **Legal Compliance:** Our project will comply with export control laws and such – FPGAs and simulation software are not of a nature that triggers special regulation for this application. We will not use any export-restricted encryption or military-grade components. All our parts and tools are standard academic/industry fare. We will also heed any lab regulations (for example, if working in a power electronics lab, normally one would have high-voltage safety procedures, but we won't be operating high voltage).

In conclusion, we identify **no serious ethical, legal, or safety issues** in this project. The work is largely simulation and digital hardware design, carried out in a controlled lab environment. The typical ethical practices of engineering research (honesty, proper attribution, and considering the broader implications of the work) are being followed. If during the project any unexpected concern arises (say, a need to test with higher power), we will conduct a risk assessment and consult with advisors before proceeding. As it stands, the project is low-risk and high-reward in demonstrating a new capability for power electronics development.

7. Conclusion and Next Steps

In this interim report, we have detailed the specifications, background theory, current progress, and future plans for the development of a real-time FPGA-based digital twin of high-frequency power converters. The project aims to push the boundaries of HIL simulation resolution using a Verlet integration solver that leverages the Hamiltonian structure of converter circuits. Our literature review confirms the novelty of this

approach and its relevance given the trend toward higher switching frequencies in power electronics ¹
27 .

So far, initial results from Python simulations of a Buck and DAB converter show that the method is numerically stable and accurate, preserving energy and handling switching events without recomputing circuit equations each step. This gives a strong foundation as we move into the hardware implementation phase. The plan laid out will guide the project through incremental milestones, mitigating risks with contingency options and aligning with academic timelines. By June, we expect to demonstrate a working FPGA-in-the-loop platform that can emulate a converter at nanosecond-scale time steps, providing a testing ground for power converter controllers that was previously unattainable with conventional tools.

Next Steps Summary: We will proceed to implement the converter models in VHDL/Verilog, validate them on the FPGA, and integrate a physical controller to close the loop. Emphasis will be on optimizing the design to truly achieve the <25 ns step, and rigorously verifying that performance. The final report will include quantitative results from the FPGA runs, comparisons with offline simulations, and a discussion on the potential for extending this approach to even more complex systems or further reducing step sizes. We are excited by the interim findings and remain on track to accomplish the project goals, potentially setting a new benchmark for real-time power electronics simulation.

Overall, this project not only serves as a capstone for our degree but also contributes to the broader engineering field by exploring how advanced numerical methods and FPGA technology can together enable ultra-high-fidelity digital twins for modern power converters. We have adhered to all technical and ethical standards in this work, and will continue to do so as we bring the project to completion. The remainder of the project will focus on executing the detailed plan and solving any practical challenges that arise, ensuring a successful demonstration by the project deadline.

1 3 6 9 16 17 18 19 20 25 31 37 38 39 40 41 44 45 46 47 48 51 55 58 62 63 64 65 66 67

68 69 70 71 72 73 74 75 84 A_Semi-Implicit_Parallel_Leapfrog_Solver_With_Half-
Step_Sampling_Technique_for_FPGA-Based_Real-Time_HIL_Simulation_of_Power_Converters.pdf
file:///file_0000000055c471fd9ec009a8e7323972

2 4 5 8 10 22 27 43 49 50 56 57 76 77 78 79 80 85 86 FPGA-Based_Submicrosecond-
Level_Real-Time_Simulation_of_Solid-State_Transformer_With_a_Switching_Frequency_of_50_kHz.pdf
file:///file_000000000e8071fdbd9a2bd525de677c

7 23 24 28 60 (PDF) Review of Real-time Simulation of Power Electronics
https://www.researchgate.net/publication/343128626_Review_of_Real-time_Simulation_of_Power_Electronics

¹¹ A real-time digital twin approach on three-phase power converters applied to condition monitoring -
ScienceDirect
<https://www.sciencedirect.com/science/article/pii/S0306261922018633>

12 52 59 FPGA-Based Digital Twins for Real-Time Monitoring of Power Converters: A Review – IJSREM
<https://ijsrem.com/download/fpga-based-digital-twins-for-real-time-monitoring-of-power-converters-a-review/>

13 14 Verlet integration - Wikipedia
https://en.wikipedia.org/wiki/Verlet_integration

15 (PDF) On the Discrete-Time Modeling of a DC-to-DC Power Converter and Control Design with Discrete-Time Sliding Modes

https://www.researchgate.net/publication/237053434_On_the_Discrete-Time_Modeling_of_a_DC-to-DC_Power_Converter_and_Control_Design_with_Discrete-Time_Sliding_Modes

21 29 A Port-Hamiltonian Perspective on Dual Active Bridge Converters: Modeling, Analysis, and Experimental Validation | MDPI

<https://www.mdpi.com/1996-1073/18/19/5197>

26 30 32 42 53 54 87 Typhoon HIL

<https://ticket.typhoon-hil.com/kb/faq.php?id=272>

33 25 ns DC-DC Solver | 2023.2 Release Tutorial - YouTube

<https://www.youtube.com/watch?v=ehN0zvKOsRI>

34 35 36 61 Integration Time Step - an overview | ScienceDirect Topics

<https://www.sciencedirect.com/topics/engineering/integration-time-step>

81 82 83 [PDF] EMTP-EMTPWorks Simulation Options

<https://www.emtp.com/documents/EMTP%20Documentation/doc/simoptions/simoptions.pdf>